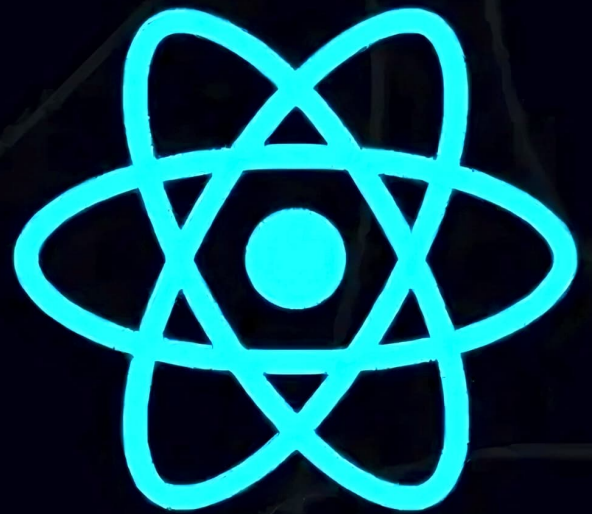


Conditional Rendering





Conditional rendering in React allows you to render different components or elements based on certain conditions. This is similar to how you use conditions in JavaScript to perform different actions.

Using if Statements

You can use if statements to **conditionally render components**. This is useful when you have more **complex conditions** or need to perform some logic before rendering.

```
import React from 'react';

function Greeting(props) {
  const isLoggedIn = props.isLoggedIn;

  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  } else {
    return <h1>Please sign up.</h1>;
  }
}

function App() {
  return (
    <div>
      <Greeting isLoggedIn={true} />
      <Greeting isLoggedIn={false} />
    </div>
  );
}

export default App;
```

In this example, the **Greeting component** checks the **isLoggedIn prop** and renders different messages based on its value.

Using Ternary Operator

The **ternary operator** is a more concise way to conditionally render elements. It's useful for simple conditions.

```
import React from 'react';

function Greeting(props) {
  return (
    <h1>
      {props.isLoggedIn ? 'Welcome back!' : 'Please sign up.'}
    </h1>
  );
}

function App() {
  return (
    <div>
      <Greeting isLoggedIn={true} />
      <Greeting isLoggedIn={false} />
    </div>
  );
}

export default App;
```

Here, the **ternary operator** `? :` is used to choose between two different strings based on the `isLoggedIn` prop.

Inline Conditional Rendering

You can also use **inline conditions** to render different components or elements

```
import React from 'react';

function App() {
  const isLoggedIn = true;

  return (
    <div>
      {isLoggedIn ? (
        <h1>Welcome back!</h1>
      ) : (
        <h1>Please sign up.</h1>
      )}
    </div>
  );
}

export default App;
```

This example uses **inline conditions** to render different elements directly within the **return** statement of the **App** component.

Summary

if statements: Best for more complex conditions or when you need to perform additional logic.

Ternary operator (`? :`): Useful for simple conditions within **JSX**.

Inline conditional rendering: Allows you to write conditions directly within the **JSX**.

These methods help you control what gets rendered in your **React components** based on different **states** or **props**, making your application more **dynamic and responsive** to user interactions.